Deploying a distributed application with Docker and Kubernetes

Distributed Systems – lab assignment

Draft v0.13 – academic year 2026–2027

1 Introduction

In the course project you build the privacy-friendly bulletin board and deploy it as a set of containers: at least 3 partition servers, 20 automated clients and 2 manual clients, all running on Docker and Kubernetes. The project uses Java RMI, and Java RMI is one of the harder protocols to run inside containers: the objects a client receives contain network addresses and port numbers, and those addresses are often wrong once the server runs in a container.

This lab prepares you for that deployment. You do not work on the bulletin board itself. Instead you get **CellStore**, a small key-value store over Java RMI that is split into partitions in the same way as the bulletin board. It works on one machine. Your job is to make it work in Docker, in Docker Compose, and finally in a Kubernetes cluster, and to make its clients survive a server that crashes and restarts.

By the end of the lab you can:

- explain how containers find and reach each other on a Docker network, and what a published port does;
- explain what an RMI stub contains and why that matters in a container;
- run a multi-container application with Docker Compose;
- deploy an application on Kubernetes with the right objects: a StatefulSet with a headless Service for servers that need a stable identity, and a Deployment for interchangeable clients;
- debug networking problems in a cluster with DNS lookups, logs and a debug container;
- make an RMI client recover from a server restart.

1.1 Organisation

Part	Where	Time	Content
Pre-lab	home, before the lab	about 0.5 h	install the tools, run CellStore locally, write the Dockerfile
Lab	lab session	3 h	Docker networking (T1), Compose (T2), Kubernetes (T3), client resilience (T4)

The pre-lab is **mandatory**. Installing Docker and a Kubernetes cluster can take an hour on its own, and there is no time for that during the lab. Bring a laptop on which everything in the pre-lab works.

1.2 What you get

A starter repository `cellstore/` containing:

```
src/ds/cellstore/
  Main.java                    entry point, dispatches on ROLE
  CellStore.java               remote interface
  CellServer.java              one partition
  CellClient.java              client: auto / interactive / verify
  Reply.java, WrongPartitionException.java, Config.java, Log.java
build.sh                       compile + package into cellstore.jar
compose.yaml                   the application in Docker Compose (T2)
k8s/                           the application in Kubernetes (T3)
verify.sh                      end-to-end test of the Kubernetes deployment (T4)
kind-config.yaml               two-node cluster for the Kubernetes part
README.md                      how to build, run and configure CellStore
```

CellStore stores string values in integer-indexed cells. With `PARTITIONS` servers, cell c belongs to partition c mod `PARTITIONS`. Every server starts its own RMI registry and binds its remote object under the name `cellstore`. A server **refuses** cells it does not own (`WrongPartitionException`), so a client that routes requests to the wrong server fails loudly instead of silently storing data in the wrong place.

The client has three modes:

- `auto`: the automated client. Forever: write a random cell, read it back, log one line (`OK`, `WRONG` or `ERROR`).
- `interactive`: the manual client. Reads `put <cell> <value>` and `get <cell>` commands from standard input.
- `verify`: a self-test. Checks that every configured server is reachable, owns the partition it is listed for, stores values, and refuses cells of other partitions. Exits with status 0 on success.

All configuration comes from environment variables; the README lists them.

2 Pre-lab (at home)

2.1 P1 – Install the tools

You need four things: Docker, kind, kubectl and a JDK 21.

Docker. On macOS and Windows install Docker Desktop, which is free for education and personal use.[1] On Windows, use the WSL 2 backend. On Linux, install Docker Engine.

- Docker Desktop for Mac: https://docs.docker.com/desktop/setup/install/mac-install/
- Docker Desktop for Windows: https://docs.docker.com/desktop/setup/install/windows-install/
- Docker Engine for Linux: https://docs.docker.com/engine/install/

kind ("Kubernetes in Docker") runs a complete Kubernetes cluster inside Docker containers on your laptop. Each container plays the role of one cluster node.

[1] See the licensing note on the installation pages. Other runtimes (OrbStack, Colima, Podman) may work, but we only support Docker Desktop and Docker Engine.

- Installation and quick start: https://kind.sigs.k8s.io/docs/user/quick-start/ (for example `brew install kind`, `winget install Kubernetes.kind`, or the release binary on Linux)

kubectl is the command-line client for every Kubernetes cluster.

- Installation for Linux, macOS and Windows: https://kubernetes.io/docs/tasks/tools/

JDK 21, to build and run CellStore outside a container. Any distribution works, for example Eclipse Temurin: https://adoptium.net/.

Now create the two-node cluster you will use during the lab, from inside the `cellstore/` directory:

```
$ kind create cluster --name ds-lab --config kind-config.yaml
$ kubectl get nodes
```

Checkpoint

`kubectl get nodes` lists two nodes, `ds-lab-control-plane` and `ds-lab-worker`, both `Ready`. Save this output: you submit it at the start of the lab.

Check that your `kubectl` matches the cluster:

```
$ kubectl version
```

The client version may differ from the server version by at most one minor version (for example client 1.36 with server 1.37). If `kubectl` warns that the difference exceeds the supported skew, update `kubectl`. On macOS and Windows, Docker Desktop installs its own copy of `kubectl`, which may be older and come first on your `PATH`; `which -a kubectl` (macOS/Linux) or `where kubectl` (Windows) lists all copies. If the right copy is listed first but `kubectl version` still reports the old version, your shell remembers the old location: run `hash -r` or open a new terminal. Version skew policy: https://kubernetes.io/releases/version-skew-policy/.

Watch out

Work in a directory whose path contains no spaces and no colons (for example `~/ds/cellstore`). Several Docker commands use `:` as a separator and break on such paths.

2.2 P2 – Run CellStore on your machine

Before moving anything into containers, you first see CellStore work on a single machine. This is the baseline: during the lab you will compare every failure against what you observe here, in particular the contents of the RMI stub.

Build the jar and start three servers and a client in four terminals, as described in the README:

```
$ ./build.sh
$ ROLE=server PARTITION=0 REGISTRY_PORT=1099 java -jar cellstore.jar   # terminal 1
$ ROLE=server PARTITION=1 REGISTRY_PORT=1100 java -jar cellstore.jar   # terminal 2
$ ROLE=server PARTITION=2 REGISTRY_PORT=1101 java -jar cellstore.jar   # terminal 3
$ ROLE=client MODE=verify java -jar cellstore.jar                      # terminal 4
```

Then run an `auto` client and an `interactive` client for a while. Stop one of the servers with Ctrl-C, watch the auto client, and start that server again. Read `CellServer.java` and `CellClient.java`; they are short.

Questions

W1. At startup each server prints the stub it hands to clients. Which IP address and port does the stub of partition 0 contain? Restart that server: what changes?

W2. List the ports your servers listen on (on macOS/Linux: `lsof -iTCP -sTCP:LISTEN -P | grep java`; on Windows: `netstat -ano`). How many ports does each server open, and which one is the registry?

W3. After you restarted a server, did the auto client recover? Explain from the code why (not).

Background on RMI stubs and registries: the Java tutorial *An Overview of RMI Applications*, https://docs.oracle.com/javase/tutorial/rmi/overview.html.

2.3 P3 – Write the Dockerfile

Write a `Dockerfile` in the root of the repository that produces **one image** for both roles: the `ROLE` environment variable decides at run time whether a container is a server or a client.

Requirements:

- Use a **multi-stage build** with two stages:
  - a *build stage* `FROM eclipse-temurin:21-jdk AS build` that copies the sources and runs `build.sh`;
  - a *runtime stage* that starts from a fresh `FROM eclipse-temurin:21-jre` and copies only the jar out of the build stage with `COPY --from=build`.

  The runtime stage must **not** start `FROM build`: a stage that starts from the build stage inherits the whole JDK and the sources. Both base images exist for amd64 and arm64 (Apple silicon).
- The image starts the jar with `java -jar`; all configuration is passed as environment variables.
- Tag the image `cellstore:1.0`.

Then check that a server container starts:

```
$ docker build -t cellstore:1.0 .
$ docker run --rm -e ROLE=server -e PARTITION=0 cellstore:1.0
```

Resources:

- Multi-stage builds: https://docs.docker.com/build/building/multi-stage/
- The `eclipse-temurin` images: https://hub.docker.com/_/eclipse-temurin

Checkpoint

`docker run` prints `partition 0/3 ready` and the stub. The runtime image contains no compiler, so `docker run --rm --entrypoint javac cellstore:1.0 -version` must *fail*; if it prints a version, your final stage is still the JDK image. Leave the image built: the lab starts from it.

Questions

W4. Compare the stub printed inside the container with the one from P2. Which address does it contain now? Can a client on your laptop reach that address? Does the answer depend on your operating system?

3 Lab session

3.1 T1 – Docker networking (about 40 minutes)

In this part you run one server (`PARTITIONS=1`) and one client, and move the client around: to another container, to a user-defined network, and to your laptop. Remove the server container between steps with `docker rm -f p0`.

Debugging tip

The client's first log line shows the server list it uses. If it ends with `(default: SERVERS is not set)`, you misspelled the variable (for example `-e SERVER=...`) and the client is talking to `localhost`, which inside a container is the client container itself.

T1.1 – The default bridge network

Containers started without `--network` are attached to Docker's default network, called `bridge` (see `docker network ls`).

```
$ docker run -d --name p0 \
    -e ROLE=server -e PARTITION=0 -e PARTITIONS=1 cellstore:1.0
$ docker logs p0
$ docker run --rm -e ROLE=client -e MODE=verify -e SERVERS=p0:1099 cellstore:1.0
```

The client cannot find `p0`. Look up the server's address and try again with that address instead of the name:

```
$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' p0
$ docker run --rm \
    -e ROLE=client -e MODE=verify -e SERVERS=<address>:1099 cellstore:1.0
```

T1.2 – A user-defined network

```
$ docker rm -f p0
$ docker network create cellnet
$ docker run -d --name p0 --network cellnet \
    -e ROLE=server -e PARTITION=0 -e PARTITIONS=1 cellstore:1.0
$ docker run --rm --network cellnet \
    -e ROLE=client -e MODE=verify -e SERVERS=p0:1099 cellstore:1.0
```

Now look inside the network with `netshoot`, a container full of networking tools. The first two commands show which DNS server a container on `cellnet` uses, and ask it for the address of `p0` (`+short` is an option of `dig`: print only the answer). The third command joins the network namespace of `p0`, so it sees exactly the sockets `p0` sees, and lists the TCP ports on which something is listening.

```
$ docker run --rm --network cellnet nicolaka/netshoot cat /etc/resolv.conf
```

```
$ docker run --rm --network cellnet nicolaka/netshoot dig +short p0
$ docker run --rm --network container:p0 nicolaka/netshoot ss -tln
```

T1.3 – A client on your laptop

Publish the registry port and run the client on your laptop (outside Docker), as in P2:

```
$ docker rm -f p0
$ docker run -d --name p0 --network cellnet -p 1099:1099 \
    -e ROLE=server -e PARTITION=0 -e PARTITIONS=1 cellstore:1.0
$ ROLE=client MODE=verify SERVERS=localhost:1099 java -jar cellstore.jar
```

The lookup reaches the registry, but the client only prints `connected` after a pause, and then the first call on the stub fails as well.

Why does it pause?

The client gives up opening a TCP connection after 5 seconds (`CONNECT_TIMEOUT_MS`). Without that limit, a connection to an address that never answers hangs for minutes. The lookup itself already waits once: as soon as the client receives a stub, RMI's distributed garbage collector contacts the address inside that stub to announce that the client holds a reference, before `lookup` returns.

T1.4 – Fixing the stub

Two things in the stub are wrong for a client on your laptop: the address and the random port. Fix both.

1. **Port.** Change `CellServer` so that the remote object is exported on a fixed port, read from a new environment variable `EXPORT_PORT` (keep 0 as the default). Rebuild the image as `cellstore:1.1` and use that tag from now on.
2. **Address.** The address in the stub comes from the system property `java.rmi.server.hostname`. You can pass JVM options to a container through the environment variable `JAVA_TOOL_OPTIONS`; the JVM confirms this with a line `Picked up JAVA_TOOL_OPTIONS`.

```
$ docker rm -f p0
$ docker run -d --name p0 --network cellnet -p 1099:1099 -p 2099:2099 \
    -e ROLE=server -e PARTITION=0 -e PARTITIONS=1 -e EXPORT_PORT=2099 \
    -e JAVA_TOOL_OPTIONS=-Djava.rmi.server.hostname=localhost cellstore:1.1
$ ROLE=client MODE=verify SERVERS=localhost:1099 java -jar cellstore.jar
$ docker run --rm --network cellnet \
    -e ROLE=client -e MODE=verify -e SERVERS=p0:1099 cellstore:1.1
```

The client on your laptop now works. The client in a container, which worked in T1.2, is now broken. Read its error message carefully.

Checkpoint

You can explain every line of `docker logs p0`, and the laptop client passes `verify` in T1.4.

Questions

N1. In T1.1, why does the name `p0` fail while the address works? The client also reached the object's random port, which you never published: why is that allowed?

N2. In T1.2, which DNS server answers the query for `p0`? `ss -tln` shows *three* listening ports. Identify each one. (Hint: look at the local address of each socket, and compare the port numbers with the stub in `docker logs p0`.)

N3. In T1.3 the registry lookup succeeded but the first call failed. Give *two* reasons why, using the stub in `docker logs p0`. (If you use Linux and the call succeeded: explain why it would still fail for a client on another machine.)

N4. In T1.4, why does the client in a container fail now? (Hint: which host and port does its error message mention, and what does `localhost` mean inside that container?) What value of `java.rmi.server.hostname` would work for *every* client of a server?

N5. Why must the published host port equal `EXPORT_PORT`? What happens with `-p 3000:2099`? (Hint: which port number does the client connect to, and where does it get that number from?)

Resources:

- Bridge networks, default vs. user-defined: `https://docs.docker.com/engine/network/drivers/bridge/`
- Port publishing: `https://docs.docker.com/engine/network/port-publishing/`
- Docker Desktop networking limitations (no route to containers from the host): `https://docs.docker.com/desktop/features/networking/networking-how-tos/`
- RMI system properties, including `java.rmi.server.hostname`: `https://docs.oracle.com/en/java/javase/21/docs/specs/rmi/properties.html`
- netshoot: `https://github.com/nicolaka/netshoot`

3.2 T2 – Docker Compose (about 25 minutes)

Starting containers one by one does not scale to 3 servers and 20 clients. Docker Compose describes the whole application in one file and starts it with one command. The starter repository contains `compose.yaml`, which runs CellStore with three servers, a scalable set of automated clients and a manual client. It uses the image `cellstore:1.1` from T1.4. Stop everything from T1 first (`docker rm -f p0`), then **read the whole file**: every setting is commented.

The Compose concepts you need:

`services`	one entry per kind of container; a service can run several identical containers
`image`, `environment`	what `docker run IMAGE` and `-e KEY=VALUE` did in T1
network	Compose creates one user-defined network for the application; every service is reachable there by its *service name* (as `p0` was in T1.2)
`depends_on`	start order: start these services first
`restart`	what Docker does when a container stops (`on-failure`: start it again)
`profiles`	services in a profile are not started by `up`, only when you name them
`--scale client=20`	run 20 containers of the `client` service

Start the application with 20 automated clients and inspect it:

```
$ docker compose up -d --scale client=20
```

```
$ docker compose ps
$ docker compose logs -f client                  # Ctrl-C stops following
$ docker compose run --rm -e MODE=verify manual  # self-test
$ docker compose run --rm manual                 # try: put 5 hello, get 5
$ docker stats --no-stream
```

Restart one server and watch what the clients do:

```
$ docker compose restart p1
$ docker compose logs --since 30s client | grep partition=1 | tail
```

Finally, find out why the `restart` policy is there. Remove the line `restart: on-failure`, start the application again from scratch, and look at the state of all client containers, including the ones that stopped:

```
$ docker compose down
$ docker compose up -d --scale client=20
$ docker compose ps -a
```

Put the line back and stop the application with `docker compose down`.

Checkpoint

`docker compose ps` shows 3 servers and 20 clients running, the client logs show `OK` lines for all three partitions, and `MODE=verify` prints `VERIFY OK`.

Questions

C1. Each server sets `java.rmi.server.hostname` to its own service name. Why that value, and not `localhost` as in T1.4?

C2. In P2 each server needed its own registry port. Why can all three servers now use port 1099 for the registry and 2099 for the object? And why does this setup need no `ports:` section at all?

C3. Without `restart: on-failure`, did all clients survive the start? Explain what `depends_on` does and does not guarantee.

C4. After `docker compose restart p1`, which exception do the clients log for partition 1? Why is it a different exception than the one you saw in W3? Do the clients recover? (Hint: a stub contains three things, see W1. Which of them are still valid after the restart?)

C5. How much memory do the 23 containers use together? What does that mean for running the minimal demo setup of the project on a laptop?

Resources:

- Compose overview and quick start: https://docs.docker.com/compose/
- Networking in Compose: https://docs.docker.com/compose/how-tos/networking/
- Startup order, `depends_on` and health checks: https://docs.docker.com/compose/how-tos/startup-order/
- Profiles: https://docs.docker.com/compose/how-tos/profiles/
- `docker compose up` and `--scale`: https://docs.docker.com/reference/cli/docker/compose/up/

3.3 T3 – Kubernetes (about 45 minutes)

Compose runs containers on one machine. Kubernetes runs them on a *cluster* of machines (nodes) and keeps them running: you describe the state you want in *manifests*, and Kubernetes creates, restarts and moves containers until the cluster matches that description. Your kind cluster from P1 has two nodes, each of which is itself a Docker container on your laptop.

The Kubernetes objects you need:

Pod	the smallest unit Kubernetes runs: one (or a few) containers with one IP address. Pods are disposable: a replacement pod gets a new IP address.
Deployment	keeps N identical, interchangeable pods running, with random names.
StatefulSet	keeps N pods running with *stable names* `<name>-0`, `<name>-1`, ...; a replacement pod gets the same name.
Service	a name for a set of pods, selected by label. A normal Service has a virtual IP that load-balances over the pods; a *headless* Service (`clusterIP: None`) has no virtual IP and gives every pod of a StatefulSet its own DNS name.
ConfigMap	key-value configuration that pods can import as environment variables.
Job	runs a pod to completion (like `docker compose run`).
labels	key-value tags on objects; Services and Deployments find their pods through them.

The starter repository contains the manifests in `k8s/`. Read all of them: every setting is commented. They correspond to `compose.yaml`:

```
k8s/servers.yaml   headless Service + StatefulSet: pods cellstore-0, -1, -2
k8s/config.yaml    ConfigMap with the SERVERS list for all clients
k8s/clients.yaml   Deployment of 20 automated clients
k8s/manual.yaml    Pod with an interactive client
k8s/verify.yaml    Job that runs the self-test
```

T3.1 – Get the image into the cluster

The cluster nodes are separate containers with their own image store: they do not see the images on your laptop. Load the image into the cluster and check that `kubectl` talks to the right cluster:

```
$ kind load docker-image cellstore:1.1 --name ds-lab
$ kubectl config current-context        # must print kind-ds-lab
```

Watch out

If you rebuild the image later under the same tag, load it again, and delete the pods that use it so that they are recreated with the new image.

T3.2 – The servers

Every server pod is created from the same template in `servers.yaml`, so the manifest cannot contain a fixed partition number or hostname per server. Instead, each pod reads its own name and index through the *downward API*: Kubernetes fills environment variables with information about the pod itself. The header of `servers.yaml` lists the DNS name of every server pod.

Your task: complete the value of `java.rmi.server.hostname` in `servers.yaml` (the line marked `TODO`). Then start the servers:

```
$ kubectl apply -f k8s/servers.yaml
```

```
$ kubectl get pods -o wide              # which node and IP address per pod?
$ kubectl logs cellstore-1              # partition and stub of one server
```

T3.3 – DNS in the cluster

Start a throw-away netshoot pod and ask the cluster DNS about the servers. `--rm` deletes the pod afterwards.

```
$ kubectl run dns --rm -it --restart=Never --image=nicolaka/netshoot -- \
    cat /etc/resolv.conf
$ kubectl run dns --rm -it --restart=Never --image=nicolaka/netshoot -- \
    dig +short cellstore.default.svc.cluster.local
$ kubectl run dns --rm -it --restart=Never --image=nicolaka/netshoot -- \
    dig +short cellstore-1.cellstore.default.svc.cluster.local
```

T3.4 – The clients

Your task: complete `SERVERS` in `config.yaml` (the line marked `TODO`). Then run the self-test and start the automated clients:

```
$ kubectl apply -f k8s/config.yaml
$ kubectl apply -f k8s/verify.yaml
$ kubectl wait --for=condition=complete job/verify --timeout=60s
$ kubectl logs job/verify
$ kubectl apply -f k8s/clients.yaml
$ kubectl get pods                      # 3 servers, 20 clients
$ kubectl logs -l app=cellstore-client --prefix --tail=2
```

To run `verify` again later, first delete the old Job with `kubectl delete job verify`.

Then try the manual client:

```
$ kubectl apply -f k8s/manual.yaml
$ kubectl attach -it manual             # type: put 5 hello, get 5, quit
```

T3.5 – Kubernetes keeps things running

Delete pods and watch Kubernetes replace them. Open a second terminal with `kubectl get pods -o wide -w` (`-w`: keep watching) and observe names, IP addresses and nodes.

```
$ kubectl delete pod <name of one client pod>
$ kubectl scale deployment cellstore-client --replicas=25
$ kubectl scale deployment cellstore-client --replicas=20
$ kubectl delete pod cellstore-1
$ kubectl logs -l app=cellstore-client --prefix --since=30s | grep partition=1 | tail
```

When you are done, remove everything with `kubectl delete -f k8s/`. The cluster itself stays for the next part.

Checkpoint

The `verify` Job prints `VERIFY OK`, `kubectl get pods` shows 3 servers and 20 clients running, and the client logs show `OK` lines for all three partitions, answered by `cellstore-0`, `-1` and `-2`.

Questions

K1. Which value did you give `java.rmi.server.hostname`, and why? Why would the pod's IP address (also available through the downward API) be a worse choice?

K2. What does DNS return for `cellstore` and for `cellstore-1.cellstore`? Why can the Java code use the short name `cellstore-1.cellstore`? (Hint: `resolv.conf`.)

K3. Why are the servers a StatefulSet and not a Deployment? And why is their Service headless? What would go wrong if clients sent their calls to a normal, load-balancing Service in front of the three servers?

K4. After `kubectl delete pod cellstore-1`: what is the same about the new pod, and what is different? What do the clients log for partition 1, and do they recover?

K5. Fill in the Kubernetes counterpart of each Compose feature: the services `p0`–`p2`; `--scale client=20`; the manual client with `profiles`; `docker compose run -e MODE=verify`; `restart: on-failure`; lookup by service name.

Resources:

- Loading an image into kind: `https://kind.sigs.k8s.io/docs/user/quick-start/`
- Pods: `https://kubernetes.io/docs/concepts/workloads/pods/`
- Deployments: `https://kubernetes.io/docs/concepts/workloads/controllers/deployment/`
- StatefulSets, stable network identity and the pod index label: `https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/`
- Services and headless Services: `https://kubernetes.io/docs/concepts/services-networking/service/`
- DNS for Services and Pods: `https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/`
- Downward API: `https://kubernetes.io/docs/concepts/workloads/pods/downward-api/`
- Using one environment variable in another (`$(VAR)`): `https://kubernetes.io/docs/tasks/inject-data-application/define-interdependent-environment-variables/`
- ConfigMaps: `https://kubernetes.io/docs/concepts/configuration/configmap/`
- Jobs: `https://kubernetes.io/docs/concepts/workloads/controllers/job/`
- kubectl quick reference: `https://kubernetes.io/docs/reference/kubectl/quick-reference/`

3.4 T4 – Surviving a server restart (about 45 minutes)

In C4 and K4 you saw that the clients never recover from a server restart: every client keeps the stub it looked up at startup, and that stub is dead. In the project the same will happen to your bulletin-board clients whenever a partition server restarts, and Kubernetes restarts pods more often than you might expect. In this part you make the client recover *by itself*, without the client process being restarted.

The starter repository contains `verify.sh`, which tests exactly that. It builds the image named in your manifests, loads it into the cluster, deploys everything from scratch, runs the `verify` Job, deletes server pod `cellstore-1`, and then checks that every automated client serves partition 1

again within 90 seconds *and* that no client was restarted. Run it once now, before you change anything; it should fail:

```
$ ./verify.sh
```

Your task: change `CellClient` so that `verify.sh` passes.

1. **Look up lazily.** `ServerHandle` must not look up the stub in its constructor: a client must start even when no server is reachable yet. `stub()` looks the stub up the first time it is needed and caches it.
2. **Forget, look up again, retry.** When a call fails because of RMI, drop the cached stub, wait, look it up again and retry the call. Limit the number of attempts, and wait longer after each failed attempt (for example 0.5, 1, 2, 4 seconds).
3. **Only retry RMI failures.** All the failures you have seen (`ConnectException`, `ConnectIOException`, `NoSuchObjectException`, `java.rmi.UnknownHostException`) are subclasses of `RemoteException`. A registry that is up before the object is bound throws `NotBoundException`, which is *not* a `RemoteException`. Do not retry `WrongPartitionException`: that is an application error, and retrying cannot fix it.
4. **Log** every retry and every new lookup, so you can see the recovery in the logs.

Then rebuild, give the image a new tag, and use that tag in *all* manifests in `k8s/` (the manifests describe what the cluster should run, so the new version goes there):

```
$ docker build -t cellstore:1.2 .
$ ./verify.sh                  # builds and loads the tag from k8s/ itself
```

Watch the recovery while `verify.sh` waits, in a second terminal:

```
$ kubectl logs -l app=cellstore-client --prefix -f | grep -E 'RETRY|LOOKUP|ERROR'
```

Checkpoint

`./verify.sh` ends with `RESULT: PASSED`. Save its complete output in `verify-output.txt`: you submit it.

Questions

R1. In which order did the clients see which exceptions during the chaos step, and how long did it take until all clients served partition 1 again?

R2. Why must the client look up the stub again instead of simply repeating the call on the stub it has?

R3. A client may retry a call that the server did execute, but whose reply was lost. For `put` and `get` in CellStore that is harmless. Is it harmless for every operation of the bulletin board of the project? (Think of an operation that removes a value from a cell.)

R4. What determines the recovery time? Name at least three factors.

Resources:

- `java.rmi.RemoteException` and its subclasses: https://docs.oracle.com/en/java/javase/21/docs/api/java.rmi/java/rmi/RemoteException.html
- DNS caching in the JVM (`InetAddress`, section "InetAddress Caching"): https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/InetAddress.html

4 Submission

Submit a Git repository (or an archive of it) containing:

- your `Dockerfile` and your changes to `CellServer.java` (T1.4) and `CellClient.java` (T4);
- `compose.yaml` and the completed manifests in `k8s/`;
- `verify-output.txt`: the complete output of a `./verify.sh` run that ends with `RESULT: PASSED`;
- `answers.md` with your answers to all questions.

We check your submission by running `./verify.sh` on a fresh kind cluster, so make sure it passes from a clean state: `kind delete cluster --name ds-lab`, create the cluster again as in P1, and run `./verify.sh`.